API Security Risk Analysis Report

- **API SECURITY RISK ANALYSIS REPORT**
- **Prepared By:**
  RUDRANSH SHARMA

- **Tools Used:**

  Postman

  GitHub

  Browser DevTools

- **API Tested:**

JSONPlaceholder

- **Date: 22/05/2026**

Introduction

This project focuses on analyzing public APIs for common security risks and vulnerabilities. APIs were tested using Postman to identify issues such as excessive data exposure, missing authentication, improper error handling, and lack of rate limiting.

The purpose of this project is to understand API security fundamentals and document findings in a professional security assessment report.

Objectives

- Analyze public API endpoints
- Identify security weaknesses
- Assess authentication mechanisms
- Check for excessive data exposure
- Evaluate rate limiting and error handling
- Recommend remediation measures

TOOLS USED

Tool	Purpose
Postman	API request and response testing
Browser DevTools	Traffic observation
GitHub	Project documentation and hosting
MS Word	Report creation

Finding 1 — Excessive Data Exposure

Endpoint Tested

GET /users

Observation

The API exposes sensitive user information including:

- email addresses

- phone numbers
- addresses

Risk Description

Sensitive information exposure may help attackers perform phishing attacks, profiling, or social engineering activities.

Severity

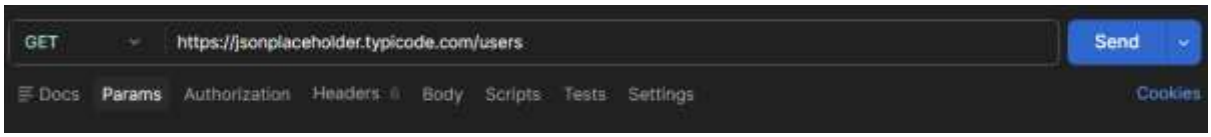
Medium

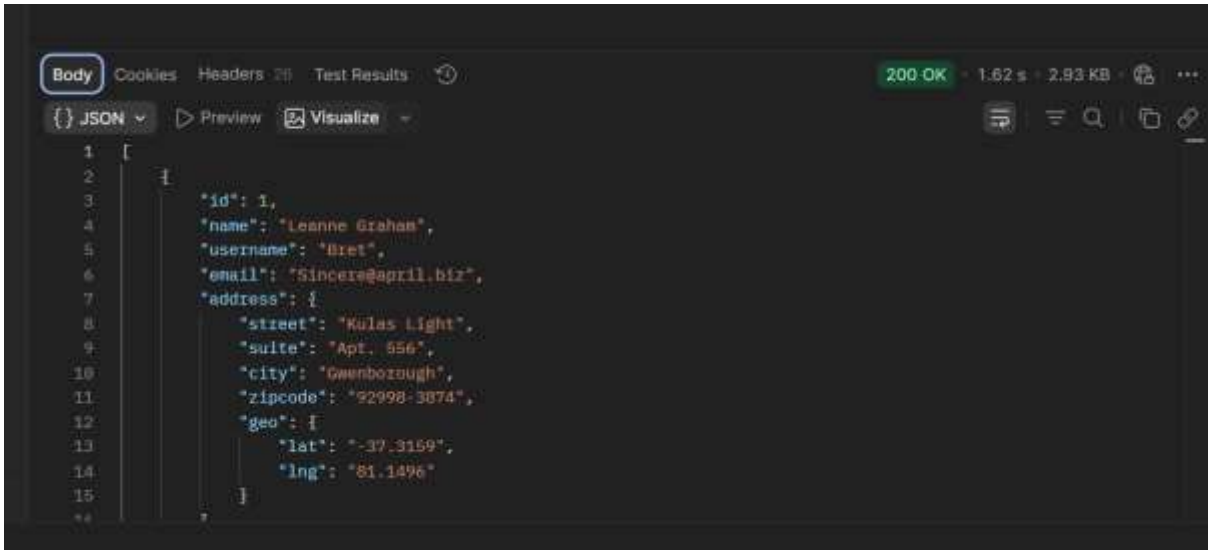
Impact

Public exposure of user information can compromise user privacy and increase cybersecurity risks.

Recommendation

Only necessary fields should be returned in API responses. Sensitive data should be restricted or masked.





Finding 2 — Public Accessibility Without Authentication

Endpoint Tested

GET /posts

Description

The API endpoint was publicly accessible without requiring authentication or authorization.

Observation

Any user could access API data directly without API keys, login credentials, or tokens.

Risk Level

Medium

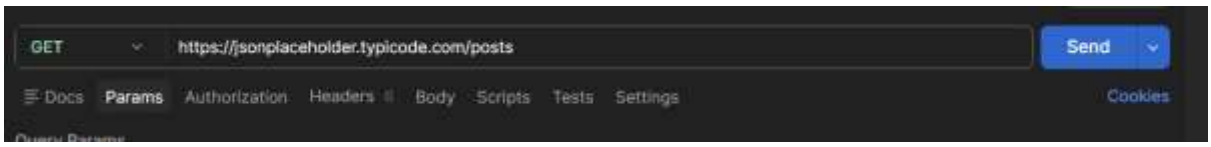
Impact

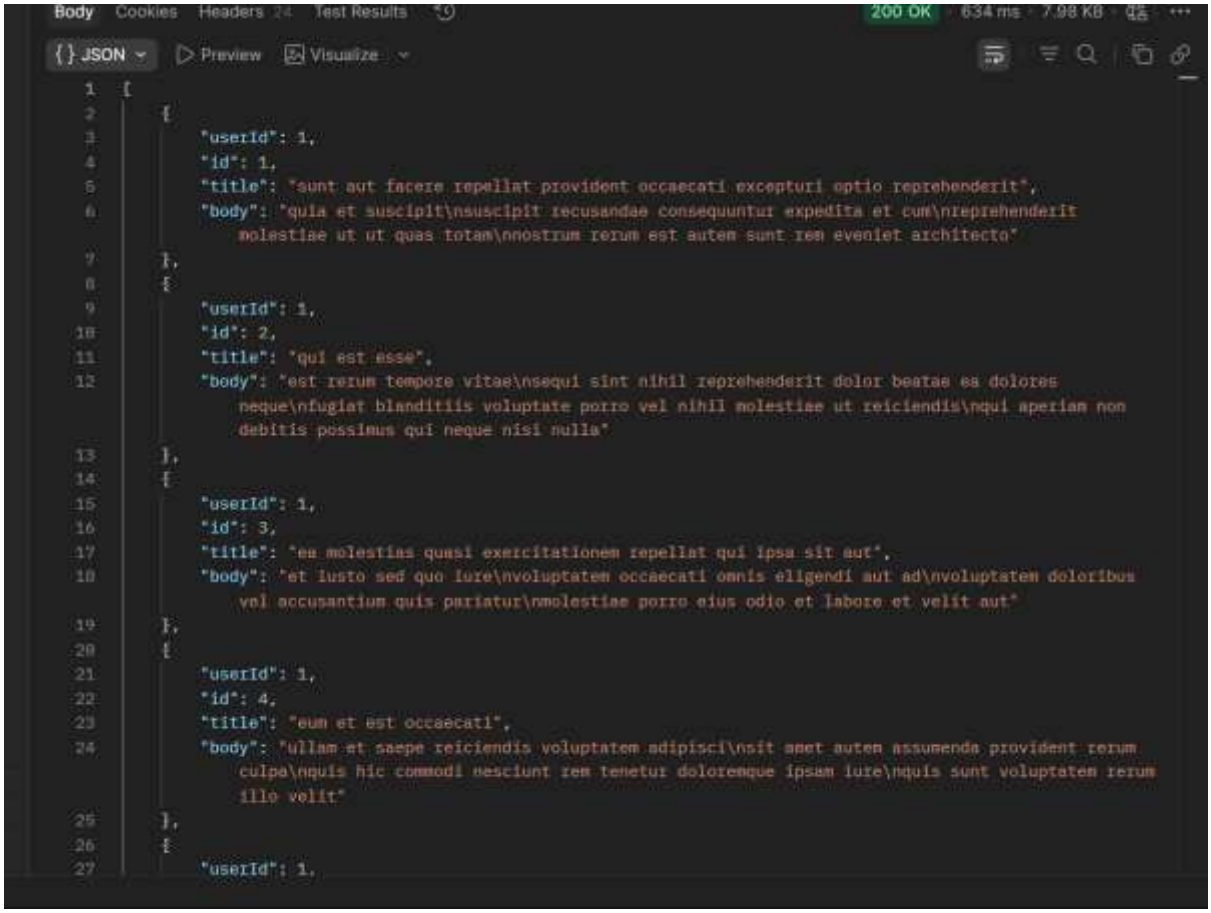
Lack of authentication may allow:

- Unauthorized data access
- Automated scraping
- Abuse of API resources

Recommendation

- Implement token-based authentication
- Restrict access to authorized users
- Use OAuth or JWT mechanisms





Finding 3 — Exposure of User Emails in Comments

Endpoint Tested

GET /comments

Description

The comments endpoint exposed email addresses associated with comments.

Observation

Email addresses were visible publicly within API responses.

Risk Level

Medium

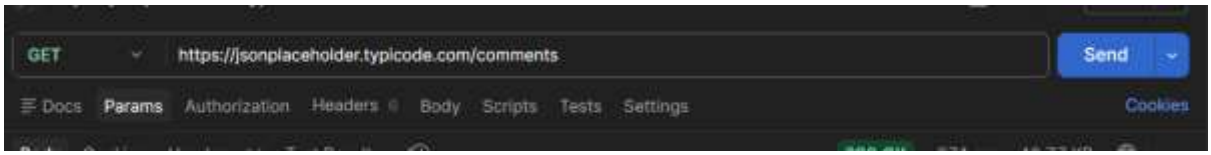
Impact

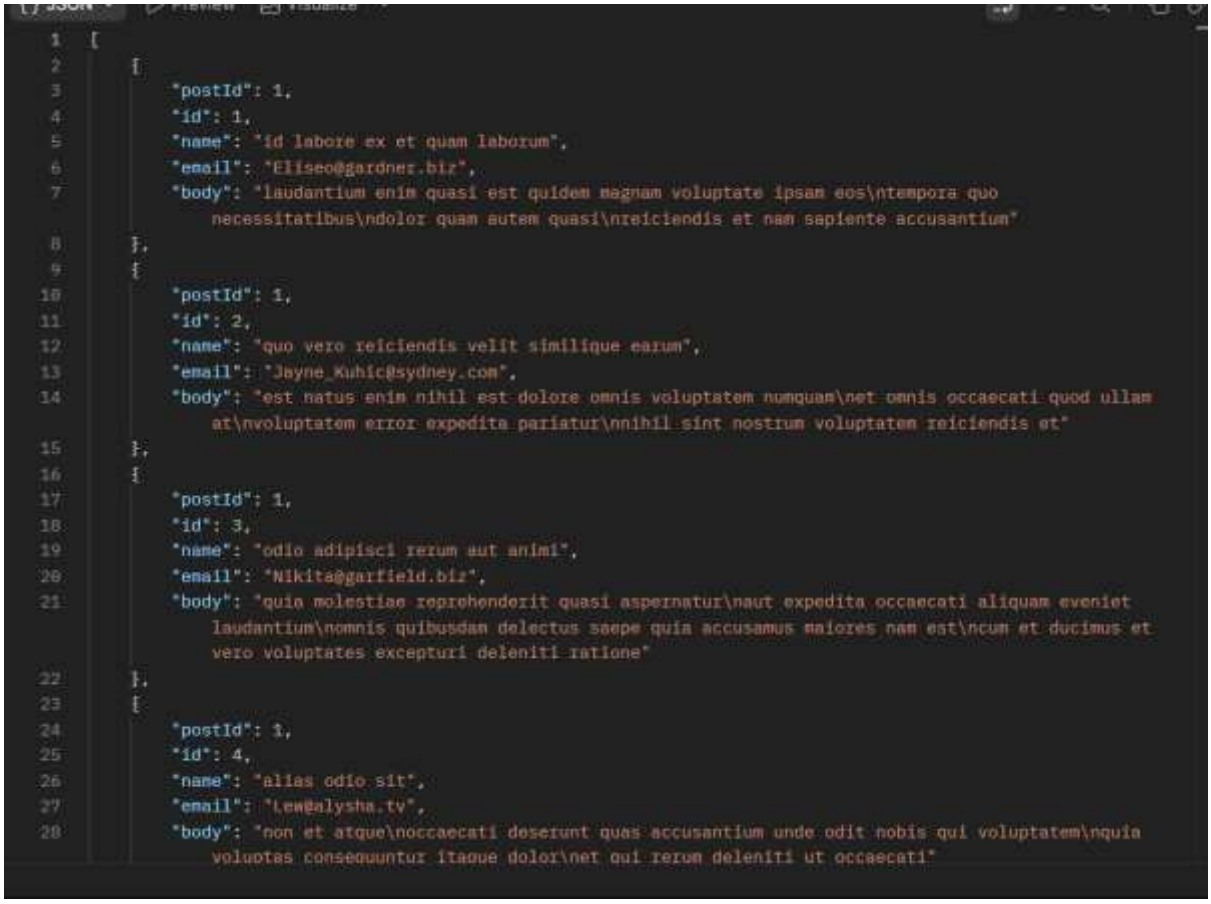
Exposed emails may lead to:

- Spam attacks
- Phishing attempts
- User tracking

Recommendation

- Mask sensitive email information
- Limit publicly exposed user data
- Apply proper privacy controls





Finding 4 — Authentication Token Generation

Endpoint Tested

POST /api/login

Description

The API generated an authentication token after successful login credentials were submitted.

Observation

The authentication mechanism successfully returned a token for valid credentials.

Risk Level

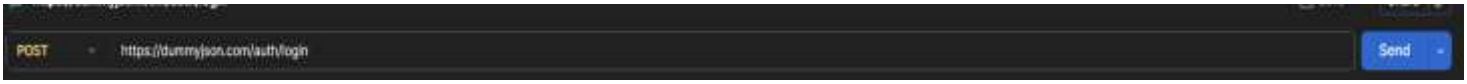
Informational

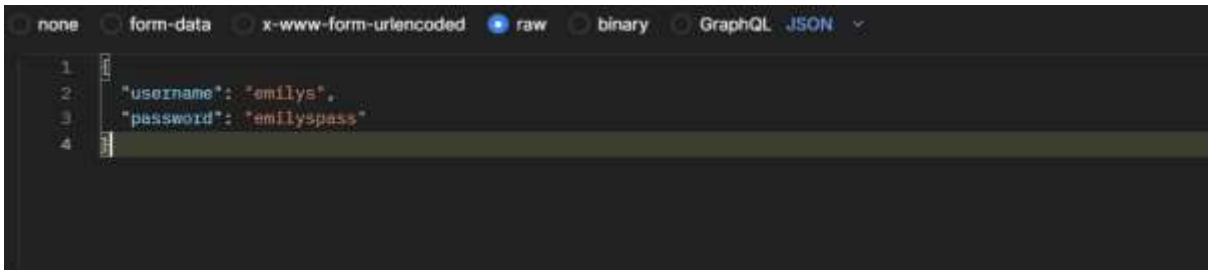
Impact

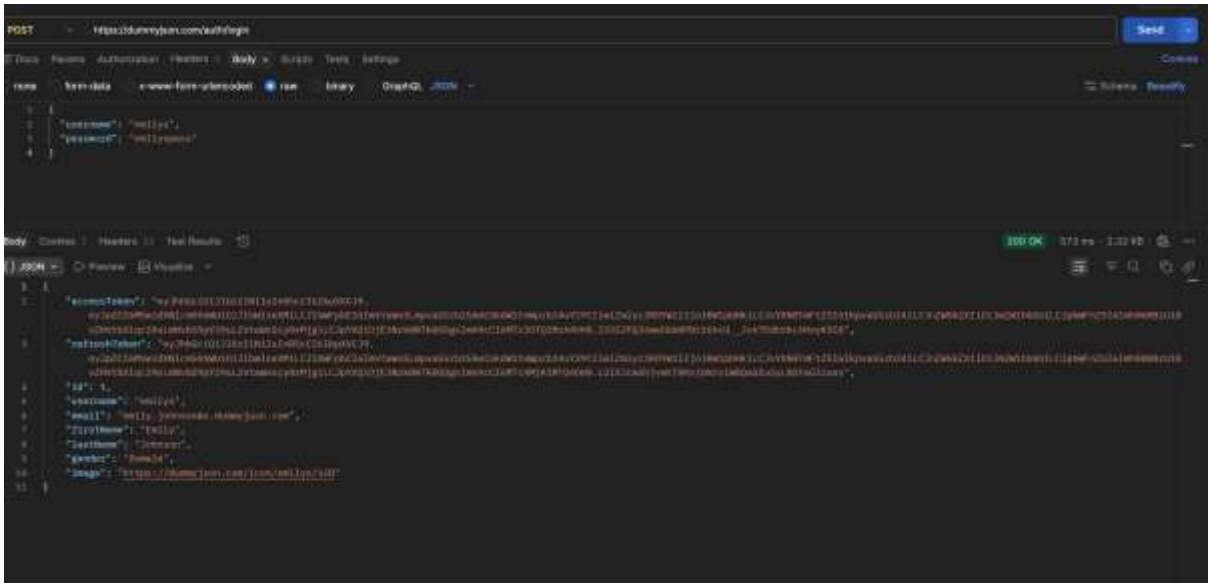
This demonstrates implementation of token-based authentication.

Recommendation

- Use secure token storage
- Apply token expiration policies
- Use HTTPS for token transmission







Finding 5 — Error Handling Analysis

Endpoint Tested

POST /auth/login

Description

The API was tested with invalid login credentials to analyze error handling behavior.

Observation

The API returned a controlled and generic error message when incorrect credentials were submitted. The response did not expose sensitive internal server details such as:

- database information
- stack traces
- server paths
- application configuration details

Risk Level

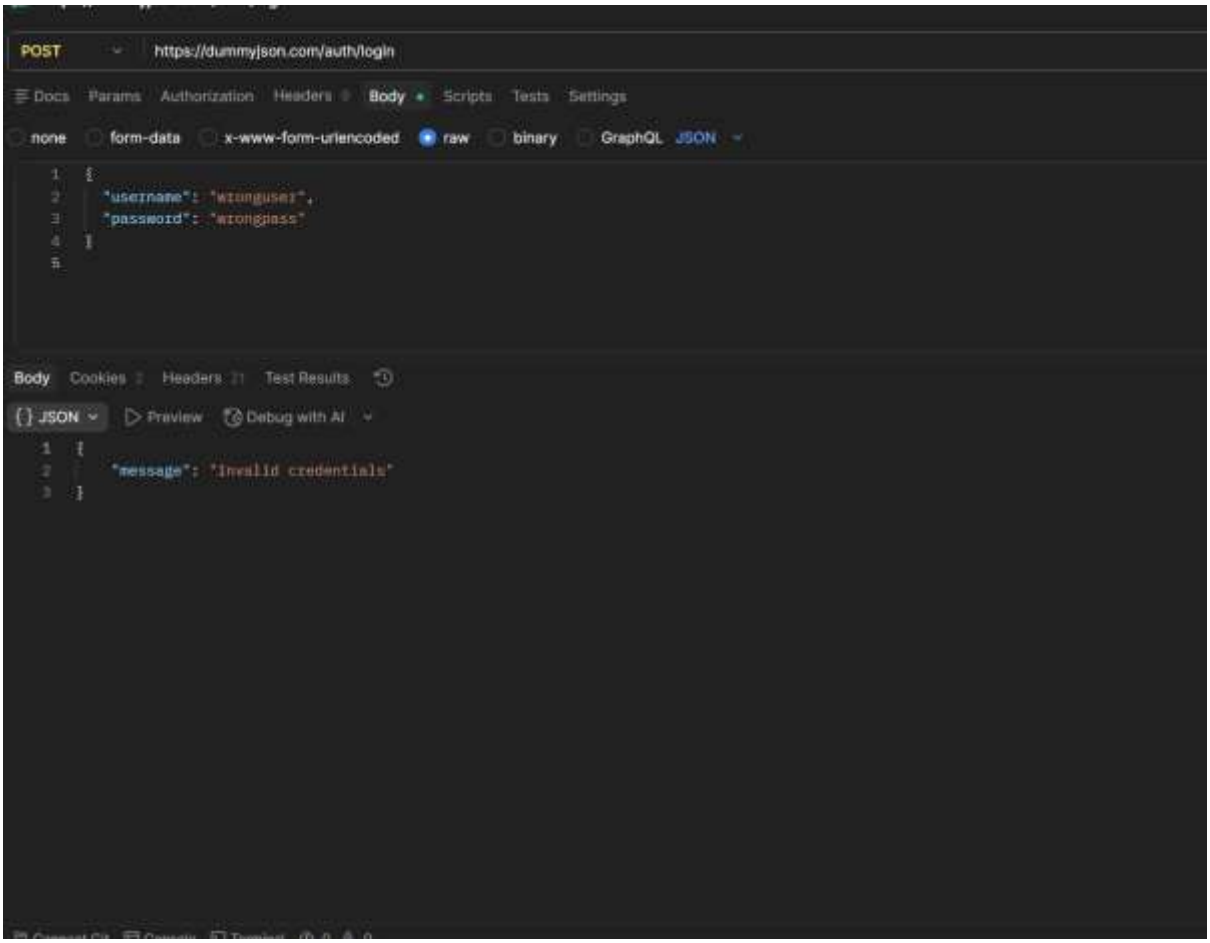
Low

Impact

Proper error handling reduces the risk of information leakage and prevents attackers from gathering internal system details that could assist in exploitation attempts.

Recommendation

- Continue using generic error messages
- Avoid exposing internal application details
- Log sensitive errors internally only



Finding 6 — Missing Rate Limiting

Endpoint Tested

GET /users

Description

The API was tested for rate limiting behavior by sending multiple repeated requests within a short period of time.

Observation

The API continued accepting repeated requests without displaying:

- request throttling
- temporary blocking
- rate limit warnings
- HTTP 429 responses

No visible rate limiting controls were observed during testing.

Risk Level

High

Impact

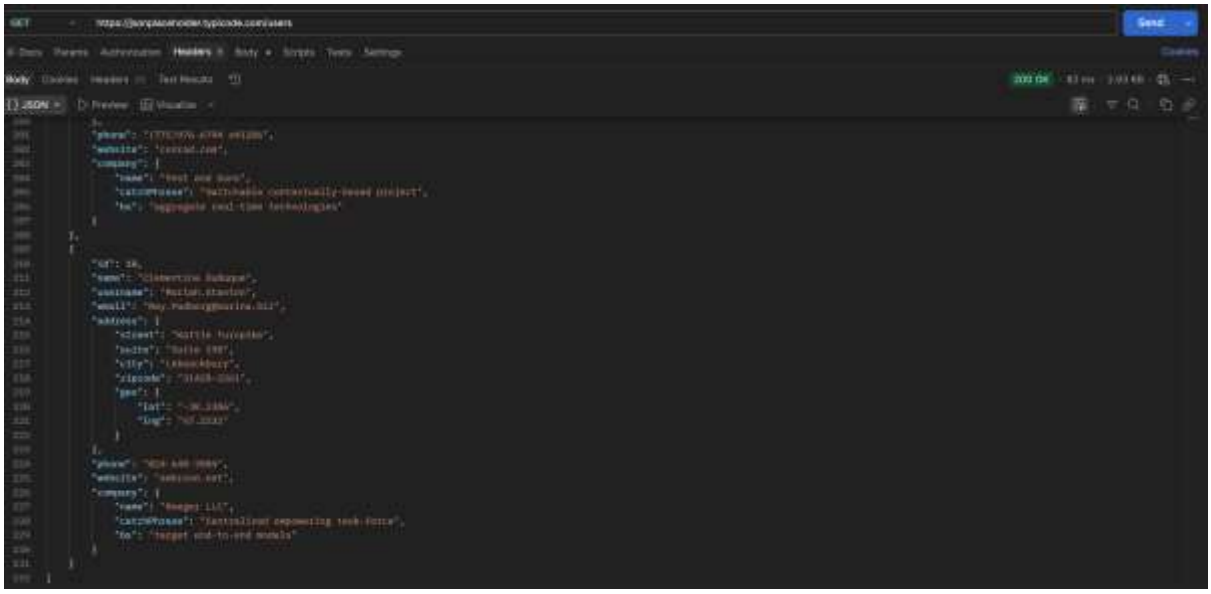
Lack of rate limiting may allow attackers to perform:

- brute-force attacks
- automated scraping
- denial-of-service attacks
- excessive API abuse

This can affect server availability and overall API security.

Recommendation

- Implement API request throttling
- Limit requests per IP address
- Configure rate limiting policies
- Use API gateways with abuse protection
- Monitor unusual traffic activity



RECOMMENDATIONS

The following security improvements are recommended:

1. Implement strong authentication mechanisms
2. Reduce unnecessary data exposure
3. Enable API rate limiting
4. Apply proper authorization checks
5. Improve input validation controls
6. Use secure token handling methods
7. Monitor suspicious API activity
8. Encrypt sensitive communications using HTTPS

Conclusion

The API Security Risk Analysis project identified common API security issues including excessive data exposure, missing authentication controls, and lack of rate limiting mechanisms.

The testing process helped in understanding practical API security concepts, authentication analysis, error handling behavior, and risk assessment methodologies.

Proper API security practices are essential to protect sensitive information and prevent misuse of application services.